大作业一项目完成报告

项目仓库

GitHub 地址：点击访问项目 GitHub 仓库

一、项目目标

本项目开发一个基于 MongoDB 的在线问卷系统，实现用户注册登录、问卷创建管理、题目配置、条件跳转、问卷填写和结果统计。

二、系统设计

后端使用 Node.js 和 Express，数据库使用 MongoDB，认证使用 JWT，密码使用 bcryptjs 加密保存，前端使用 HTML、CSS 和 JavaScript。

系统主要模块：

1. 用户认证模块
2. 问卷管理模块
3. 题目和校验模块
4. 条件跳转模块
5. 问卷填写模块
6. 统计分析模块

三、MongoDB 设计

users 集合

保存用户名、密码哈希、创建时间和更新时间。

surveys 集合

保存问卷所有者、标题、说明、访问码、状态、截止时间、题目数组和跳转规则数组。

responses 集合

保存问卷编号、填写用户、是否匿名、回答数组和提交时间。

选择 MongoDB 的原因：

- 问卷题目类型具有多样性
- 不同题目的校验字段不同
- 跳转规则适合嵌入问卷文档
- 文档结构可以减少复杂关联查询
- MongoDB 便于后续扩展题目类型

字段与索引说明

users 集合

字段	类型	说明
`_id`	ObjectId	用户主键
`username`	String	用户名，唯一索引
`passwordHash`	String	bcrypt 密码哈希
`createdAt`	Date	注册时间
`updatedAt`	Date	修改时间

surveys 集合

字段	类型	说明
`_id`	ObjectId	问卷主键
`ownerId`	ObjectId	创建者编号，建立索引
`title`	String	问卷标题
`accessCode`	String	公开访问码，唯一索引
`status`	String	draft、published 或 closed
`deadline`	Date	截止时间
`questions`	Array	嵌入式题目文档
`jumpRules`	Array	嵌入式跳转规则文档

responses 集合

字段	类型	说明
`_id`	ObjectId	提交记录主键
`surveyId`	ObjectId	所属问卷，建立索引

字段	类型	说明
`respondentId`	ObjectId/null	登录用户编号
`anonymous`	Boolean	是否匿名提交
`answers`	Array	题目答案数组
`submittedAt`	Date	提交时间，建立索引

嵌入与引用选择

题目和跳转规则嵌入 `surveys` 文档，因为它们只属于某一份问卷，读取问卷时通常需要同时读取这些内容，嵌入可以减少关联查询。

问卷所有者、提交用户和提交记录使用引用关系，因为用户和回答记录可能独立增长，并且统计时需要按问卷筛选大量提交数据。这样可以避免问卷文档无限增长，也便于对回答记录建立索引。

MongoDB 索引

- `users.username`：保证用户名唯一并加快登录查询
- `surveys.ownerId`：加快查询当前用户创建的问卷
- `surveys.accessCode`：加快公开访问问卷
- `surveys.status`：加快筛选已发布问卷
- `responses.surveyId`：加快问卷统计查询
- `responses.respondentId`：支持按填写用户查询记录
- `responses.submittedAt`：支持按提交时间排序

MongoDB 文档示例

users 文档

```
{
  "_id": "ObjectId",
  "username": "testuser",
  "passwordHash": "$2b$12$...",
  "createdAt": "2026-10-15T12:00:00.000Z",
  "updatedAt": "2026-10-15T12:00:00.000Z"
}
```

surveys 文档

```
{
  "_id": "ObjectId",
  "ownerId": "ObjectId",
  "title": "学生信息调查问卷",
  "description": "用于测试问卷功能",
  "accessCode": "2b5ddbf18f84",
```

```
  "allowAnonymous": true,
  "status": "published",
  "questions": [
    {
      "_id": "ObjectId",
      "type": "single",
      "title": "你的年级",
      "required": true,
      "options": [
        { "label": "大一", "value": "大一" },
        { "label": "大二", "value": "大二" }
      ]
    }
  ],
  "jumpRules": [
    {
      "sourceQuestionId": "ObjectId",
      "operator": "equals",
      "expectedValue": "大一",
      "targetQuestionId": "ObjectId"
    }
  ]
}
```

responses 文档

```
{
  "_id": "ObjectId",
  "surveyId": "ObjectId",
  "respondentId": null,
  "anonymous": true,
  "answers": [
    {
      "questionId": "ObjectId",
      "value": "大一"
    },
    {
      "questionId": "ObjectId",
      "value": 20
    }
  ],
  "submittedAt": "2026-10-15T12:30:00.000Z"
}
```

系统业务流程

1. 用户注册或登录，后端返回 JWT。
2. 用户创建草稿问卷，系统生成唯一访问码。
3. 创建者添加题目、校验规则和条件跳转规则。

4. 问卷发布后，填写者通过访问码获取公开问卷。
5. 填写者按跳转规则逐题回答，系统实时进行答案校验。
6. 提交成功后，回答保存到 `responses` 集合。
7. 创建者通过统计接口查看回答数量、选项分布、文本内容和数字平均值。

四、关键业务逻辑

用户认证

注册时使用 bcryptjs 对密码进行哈希，登录成功后签发 JWT。需要权限的接口通过 Bearer Token 验证用户身份。

问卷管理

用户可以创建草稿问卷，添加题目和跳转规则。问卷至少包含一道题目后才能发布。问卷发布后可以通过访问码公开访问，关闭后不能继续填写。

题目类型

系统支持：

- 单选题
- 多选题
- 文本填空题
- 数字填空题

校验逻辑

系统支持必答校验、选项合法性校验、多选数量校验、文本长度校验、数字范围校验和整数校验。

跳转逻辑

跳转规则由问卷创建者动态配置。系统根据当前题目答案和比较运算符查找匹配规则，并返回下一道题目。

统计逻辑

统计接口根据已提交的回答动态生成：

- 单选题各选项数量
- 多选题各选项选择次数
- 文本题全部文本内容

- 数字题平均值和全部数字

五、测试结果

执行命令：

```
Set-Location .\backend
npm test
```

测试结果：

所有自动化测试通过

测试覆盖：

- 用户注册
- 创建问卷
- 添加题目
- 添加跳转规则
- 发布问卷
- 获取公开问卷
- 条件跳转
- 提交问卷
- 查看统计

失败场景测试

- 重复用户名注册：正确返回失败
- 无效单选答案：正确拒绝提交
- 未填写必答题：正确拒绝提交
- 非法数字答案：正确拒绝提交
- 多选数量超限：正确拒绝提交
- 未授权统计请求：正确拒绝访问
- 已关闭问卷提交：正确拒绝提交
- 超过截止时间提交：正确拒绝提交

实际功能演示结果

- 成功创建“学生信息调查问卷”
- 成功添加单选题“你的年级”
- 成功添加数字题“你的年龄”
- 成功配置“大一跳转到年龄题”

- 成功发布问卷并生成访问地址
- 成功按跳转规则完成填写
- 成功提交问卷
- 统计结果显示总回答数为 1
- 年级“大一”选择数为 1
- 年龄平均值为 20

完整测试用例、输入、预期输出和实际结果见 `TEST_CASES.md`。

六、遇到的问题及解决方法

1. npm 项目名称问题

项目根目录包含中文名称，直接执行 `npm init -y` 时失败。解决方法是在 `backend` 子目录中初始化 Node.js 项目。

2. MongoDB 服务未注册为 Windows 服务

系统中没有 MongoDB Windows 服务。解决方法是使用本地数据库目录手动启动：

```
mongod --dbpath .\backend\data\db
```

3. 自动化测试请求缺少 JSON 请求头

测试工具合并请求配置时覆盖了 `Content-Type`。调整请求配置合并顺序后解决。

4. nodemon 频繁重启

开发期间使用 `node src/server.js` 直接启动服务，避免 nodemon 监控文件造成重复重启。

七、AI 使用说明

AI 主要帮助完成：

- 数据库集合和字段设计
- Express 路由实现
- JWT 认证中间件
- 问卷校验和跳转逻辑
- 统计接口
- 自动化测试代码
- 项目说明文档

人工完成和修改：

- 根据课程要求确定功能范围
- 检查并调整 MongoDB 数据结构
- 增加权限和匿名填写逻辑
- 修复测试请求头问题
- 验证全部 API 流程
- 编写启动说明和测试说明

八、项目运行方式

1. 启动 MongoDB。
2. 在 `backend` 目录执行 `node src/server.js`。
3. 在 `frontend` 目录执行 `python -m http.server 5500`。
4. 浏览器访问 `http://localhost:5500`。

九、迭代改进记录

在基础功能完成后，对系统进行了以下改进：

1. 多选题禁止提交重复选项。
2. 单选题和多选题禁止使用空选项值或重复选项值。
3. 增加多选题、文本题和数字题的限制关系校验。
4. 跳转规则增加题型匹配校验。
5. 禁止跳转到当前题目自身。
6. 禁止重复添加相同跳转规则。
7. 无效或过期 JWT 不再降级为匿名提交。
8. 问卷截止时间增加创建、修改、发布和下一题接口校验。
9. 注册时处理 MongoDB 用户名唯一索引冲突。
10. 移除问卷创建接口中的服务器堆栈信息泄露。
11. 严格校验 `allowAnonymous` 必须为布尔值。
12. 增加了对应的自动化测试用例，覆盖重复选项、非法范围、无效 Token、非法截止时间、非法匿名配置和重复跳转规则等场景。

所有改进完成后，执行 `npm test`，自动化测试全部通过。

十、阶段一后续改进记录

在初步测试完成后，项目继续进行了以下改进：

1. 扩充登录测试

新增了正确登录和错误密码登录测试，验证：

- 正确用户名和密码可以获得 JWT
- 错误密码返回失败
- 登录返回的用户信息与请求用户名一致

2. 扩充权限测试

新增了越权查看统计测试，验证：

- 问卷创建者可以查看自己的统计结果
- 其他用户不能查看该问卷统计

3. 扩充问卷状态测试

新增了以下测试：

- 非匿名问卷禁止未登录用户提交
- 关闭问卷后禁止继续提交
- 超过截止时间后禁止提交

4. 扩充答案校验测试

新增了以下测试：

- 未知题目 ID 被拒绝
- 同一道题不能重复提交答案
- 多选题最少选择数量校验
- 多选题最多选择数量校验
- 文本题最小长度校验
- 文本题最大长度校验
- 数字题最小值校验
- 数字题最大值校验
- 数字题整数校验

5. 补充条件跳转测试

新增测试验证：

- 跳转后被跳过的必答题不会阻塞提交

- 实际可达路径上的必答题仍然必须填写

6. 前端安全性改进

对前端动态内容进行了安全处理：

- 统计页面对问卷标题、题目标题、选项和文本答案进行 HTML 转义
- 首页问卷列表对问卷标题和访问码进行 HTML 转义
- 管理页面对题目标题和题目 ID 进行 HTML 转义
- 填写页面使用 DOM API 创建单选和多选选项，避免直接拼接用户输入 HTML

7. 后端答案完整性校验

提交问卷时增加：

- 未知题目 ID 校验
- 同一道题重复提交校验
- 缺少题目 ID 校验

这些改进使测试文档、自动化测试和实际代码行为更加一致，并降低了前端动态内容产生 HTML 注入风险的可能性。

十一、重复填写控制与浏览器自动化测试

1. 重复填写控制

为了支持更真实的问卷业务场景，系统增加了 `allowMultipleSubmissions` 字段：

- `true`：允许同一登录用户多次提交
- `false`：同一登录用户只能提交一次

该字段保存在 `surveys` 集合中，默认值为 `true`，保证原有问卷行为兼容。

当用户提交问卷时，后端会根据以下条件查询历史回答：

- 当前问卷 ID
- 当前登录用户 ID

如果问卷禁止重复提交且已经存在历史回答，接口返回：

409 你已经提交过该问卷，不能重复提交

匿名用户暂不执行重复提交限制，因为匿名请求没有稳定的用户身份标识。

2. 前端配置

首页创建问卷时可以设置：

- 是否允许匿名填写
- 是否允许同一用户重复填写

管理页面加载问卷后，可以查看和修改这两个设置。

3. 浏览器端自动化测试

项目增加了基于 Playwright 的浏览器端冒烟测试，文件为：

backend/tests/browser-smoke.test.js

测试内容包括：

- 首页加载
- 填写问卷页面加载
- 管理问卷页面加载
- 统计页面加载
- 重复填写配置控件存在
- 统计页面容器存在

测试命令：

Set-Location .\frontend python -m http.server 5500

Set-Location .\backend
$env:FRONTEND_BASE_URL="http://127.0.0.1:5500/frontend" npm run test:browser

测试结果：

浏览器端冒烟测试通过

4. 新增依赖

浏览器测试使用：

playwright Chromium

依赖安装命令：

npm install --save-dev playwright npx playwright install chromium

通过 API 自动化测试和浏览器端自动化测试相结合，项目同时验证了后端业务逻辑和前端页面基本可用性。

5. 浏览器端端到端流程测试

在浏览器冒烟测试的基础上，项目增加了完整的浏览器端端到端自动化测试，测试文件为：

backend/tests/browser-e2e.test.js

该测试使用 Playwright 和 Chromium，模拟真实用户完成以下完整业务流程：

注册用户 → 创建问卷 → 设置匿名填写和重复填写选项 → 添加单选题 → 发布问卷 → 打开公开填写链接 → 选择答案 → 提交问卷 → 查看问卷统计。

测试具体步骤：

1. 打开首页并注册新的测试用户。
2. 等待登录成功并进入问卷管理区域。
3. 创建一份新的问卷。
4. 设置问卷标题、说明、允许匿名填写和允许重复填写。
5. 读取创建成功后的问卷 ID。
6. 进入问卷管理页面。
7. 添加一道必答单选题。
8. 发布问卷并读取公开访问码。
9. 打开公开填写页面。
10. 选择单选题答案。
11. 提交问卷。
12. 打开统计页面。
13. 验证总回答数为 1，并验证统计结果中包含测试题目。

执行端到端测试前，需要启动 MongoDB、后端服务和前端静态服务器。

前端静态服务器可以从项目根目录启动：

```
Set-Location "D:\Users\师旭辉\Desktop\数据库\大作业 1"
python -m http.server 5500
```

后端服务使用 3000 端口：

```
Set-Location "D:\Users\师旭辉\Desktop\数据库\大作业 1\backend"
$env:PORT="3000"
```

```
$env:MONGODB_URI="mongodb://127.0.0.1:27017/questionnaire_system"
$env:JWT_SECRET="questionnaire-local-development-secret"
node src/server.js
```

另开一个 PowerShell 窗口执行端到端测试：

```
Set-Location "D:\Users\师旭辉\Desktop\数据库\大作业 1\backend"
$env:FRONTEND_BASE_URL="http://127.0.0.1:5500/frontend"
npm run test:e2e
```

测试成功输出：

```
浏览器端到端流程测试通过
```

该测试验证了前端页面、后端接口、JWT 登录认证、MongoDB 数据写入、问卷发布、公开填写、答案提交和统计查询之间的完整业务链路。

测试过程中还修复了以下问题：

- 前端静态服务器从项目根目录启动时，测试地址必须包含 /frontend 路径。
- 后端服务必须运行在 3000 端口，确保前端 API 地址正确。
- 创建问卷后需要等待页面重新加载和问卷列表刷新。
- 前端切换题目时会重新渲染提交按钮，自动化测试需要等待按钮真正可用。
- 提交按钮点击可能触发页面跳转，测试需要处理页面执行环境被销毁的正常情况。

最终端到端测试执行结果：

```
浏览器端到端流程测试通过
```